

This member-only story is on us. [Upgrade](#) to access all of Medium.

★ Member-only story

How to Fine-Tune Newly Released LLama-3.2–11B Vision-Language Models (VLMs) on Custom Dataset.



Samar Singh · [Follow](#)

10 min read · Oct 3, 2024



21

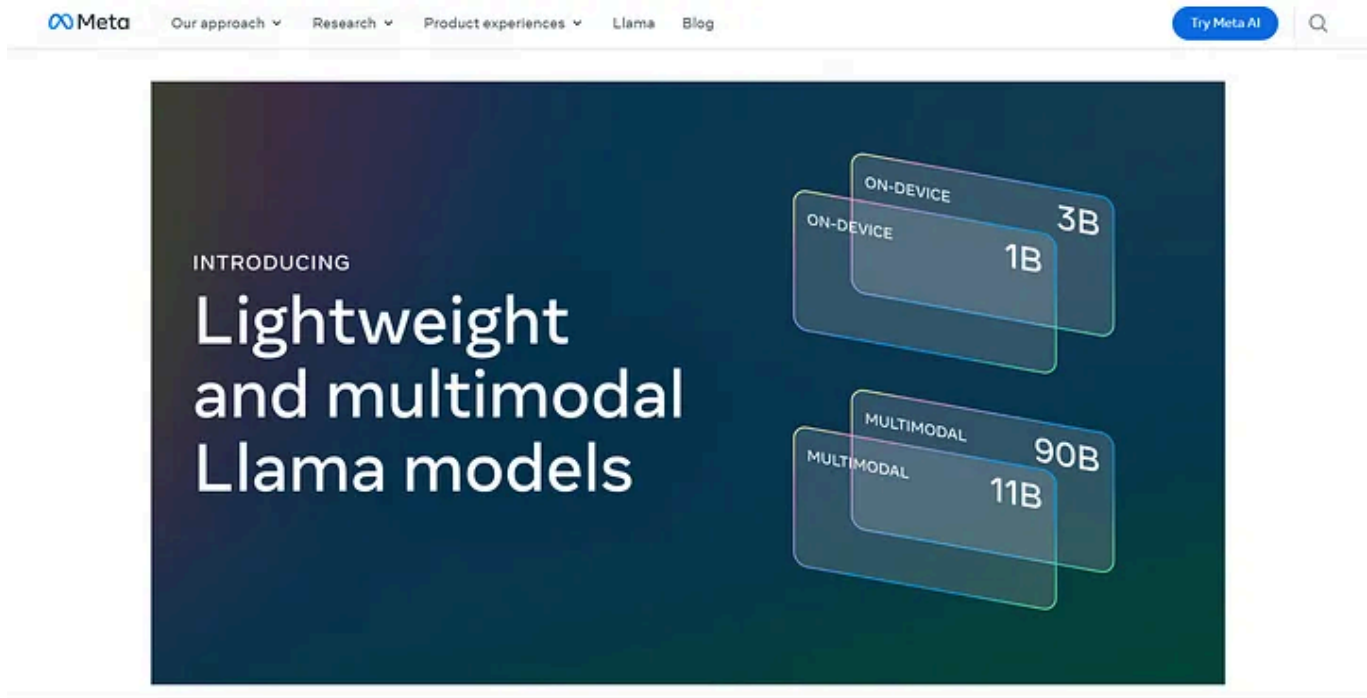


2



Multimodal large language models (LLMs) are pushing the boundaries of AI capabilities by enabling machines to understand both visual and textual information. Popular models such as Meta AI's Llama-3.2–11B-Vision, Mistral AI's Pixtral-12B and Qwen's Qwen2-VL-7B can handle tasks like image captioning, visual question answering, and image-text matching out of the box.

While these pre-trained models are impressive, sometimes you need to fine-tune them to suit your specific needs. This guide will walk you through how to fine-tune these powerful VLMs, specifically newly released llama 3.2 vision model.



Key Steps in Fine-Tuning a VLM:

1. Define your use case
2. Set up the development environment
3. Prepare the dataset
4. Fine-tune the VLM using TRL and SFTTrainer
5. Test the fine-tuned model

Let's dive into each step in detail.

1. Defining Your Multimodal Use Case

Before fine-tuning any model, you must have a clear understanding of the problem you're trying to solve. For VLMs, this typically involves tasks that integrate visual and textual data. Sometime we don't need to fine tuned because model is capable enough to generate output based on the input. So

first you need to test the model. Based on that you can take decision you need to fine tune or not.

Example Use Case:

Imagine you're building an e-commerce platform where sellers upload product images and metadata, and you need a system that automatically generates detailed product descriptions based on this input. The aim is to help sellers create better listings faster and more consistently. To achieve this, you want to fine-tune a VLM on a dataset that combines product images and metadata with high-quality product descriptions.

Why Fine-Tune?

Though pre-trained VLMs may perform well, they may not fully capture the unique attributes of your dataset or use case. LLama-3.2 VLM is a powerful Vision-Language Model that integrates the strengths of language models and vision processing. Fine-tuning allows you to adapt the model to your specific needs, such as generating SEO-optimized product descriptions or performing advanced image-based tasks.

Note: Llama 3.2 11B Vision requires at least 24 GB of GPU memory for efficient training or fine-tuning. Larger memory (32 GB or 40 GB) would be more ideal, especially if you're performing tasks with large batch sizes or complex datasets.

If you don't have the GPU and don't wanna fine tuned. Than Read it and understand it for the knowledge. I will try my best to explain each and every code in simple way. Code notebook link is given at the end. Let's move to the next step.

2. Setting Up the Development Environment

Essential Libraries:

You'll need to install key libraries to work with Hugging Face models, datasets, and TRL for fine-tuning:

```
!pip install "torch==2.4.0" tensorboard pillow torchvision accelerate huggingface
!pip install --upgrade \
    "transformers==4.45.1" \
    "datasets==3.0.1" \
    "accelerate==0.34.2" \
    "evaluate==0.4.3" \
    "bitsandbytes==0.44.0" \
    "trl==0.11.1" \
    "peft==0.13.0" \
    "qwen_vl_utils"
```

Explanation:

- **torch==2.4.0:** Installs PyTorch, the deep learning framework needed for training models.
- **tensorboard:** A tool to visualize training metrics, such as loss, accuracy, etc.
- **pillow:** A Python library for image processing (loading images for the model).
- **torchvision:** Provides datasets, models, and transforms specifically for computer vision tasks.
- **accelerate:** A Hugging Face utility to handle multi-GPU training with ease.

- **huggingface_hub**: A library to manage Hugging Face model repositories, datasets, and more.
- **transformers==4.45.1**: This installs Hugging Face's `transformers` library to load and use large language models like LLama-3.2.
- **datasets==3.0.1**: Provides access to Hugging Face datasets, enabling easy data handling.
- **evaluate==0.4.3**: A library for evaluating model performance.
- **bitsandbytes==0.44.0**: Optimizes memory usage and enables quantization to make large models more efficient.
- **trl==0.11.1**: A library used for fine-tuning large language models using techniques like LoRA.
- **peft==0.13.0**: Short for "Parameter-Efficient Fine-Tuning," it's a library for applying methods like LoRA.
- **qwen_vl_utils**: A utility package to help with processing image and vision inputs for the model

Hugging Face Hub:

You'll need to log in to Hugging Face to access the model and dataset. To use Llama 3.2 vision model, [You need to agree to share your contact information to access this model](#). And during creating your api key don't forget to select the llama3.2 vision model as in below image.

Repositories permissions

Override any user-level or org-level permissions set below for the specified repositories. The token will always have read access to all public repos contents.

meta-llama/llama-3.2-11B-Vision-Instruct ×

- ☒ Read access to contents of selected repos
- ☐ Interact with discussions / Open pull requests on selected repos
- ☐ Write access to contents/settings of selected repos

Add your API token here:

```
from huggingface_hub import login

login(token="YOUR_HF_TOKEN")
```

3. Preparing the Dataset

In this example, we're using the Amazon Product Descriptions VLM dataset. It contains images and metadata about products, which we'll use to train our LLama-3.2 vision model. This tutorial is inspired by [Philipp Schmid tutorial](#) in that he used Qwen2-VL model to fine tune. I have just made some changes according to llama 3.2 Vision model. I am using his dataset. If you want to use your custom dataset. You can use that one in the same format. To prepare the dataset you can use chatgpt or gemini model as he did.

Dataset Viewer

Split (1)
train · 1.35k rows

Search this dataset

image	Uniq Id	Product Name	Category	Selling Price	Model Number
image · width (px)	string · lengths	string · lengths	string · lengths	string · lengths	string · lengths
	00cb3b00482712567c2100767ec28a6a	Baible Fashionistas Doll	Toys & Games Dolls &	\$15.99	FJF44
	00cce525ebf9181ebfba30dc5ca936fd	Redcat Racing Aluminum Rear	Toys & Games Hobbies Remote	\$14.40	06049B
	015cc42a0e93b15bcee9425d63ecbbd9	Tru-Ray Heavyweight	Arts, Crafts & Sewing Crafting	\$10.10	6592
	018a24a463d09f047a7413603597202c	Shapes Structured Vinyl Soft Seatin	Home & Kitchen Furniture Kids	\$146.76	SPG-1000BL-A

Downloads last month: 81

Size of downloaded dataset files: 47.6 MB

Size of the auto-converted Parquet files: 47.6 MB

Number of rows: 1,345

note the image is not provided in the prompt its included as part of the "proc
prompt= ""Create a Short Product description based on the provided ##PRODUCT NAME
Only return description. The description should be SEO optimized and for a better

```
##PRODUCT NAME##: {product_name}
##CATEGORY##: {category}"""
```

```
from datasets import load_dataset
```

```
# Convert dataset to OAI messages
```

```
def format_data(sample):
```

```
    return {"messages": [
```

```
        {
```

```
            "role": "user",
```

```
            "content": [
```

```
                {
```

```
                    "type": "text",
```

```
                    "text": prompt.format(product_name=sample["Product Name"])
```

```
                }, {
```

```
                    "type": "image",
```

```
                    "image": sample["image"],
```

```
                }
```

```

        ],
    },
    {
        "role": "assistant",
        "content": [{"type": "text", "text": sample["description"]}]]
    },
],
}

# Load dataset from the hub
dataset_id = "philschmid/amazon-product-descriptions-vlm"
dataset = load_dataset("philschmid/amazon-product-descriptions-vlm", split="train")

# Convert dataset to OAI messages
# need to use list comprehension to keep Pil.Image type, .map convert image to
dataset = [format_data(sample) for sample in dataset]

print(dataset[345]["messages"])

```

```

warnings.warn(
README.txt 100% ██████████ 1.22M/1.22M [00:00<00:00, 83.0kB/s]
train 00000 of 00001 parquet: 100% ██████████ 47.6M/47.6M [00:02<00:00, 23.2MB/s]
Generating train split: 100% ██████████ 1345/1345 [00:00<00:00, 6054.04 examples/s]
[{"role": "system", "content": [{"type": "text", "text": "You are an expert product description writer for Amazon."}]}, {"role": "user", "content": [{"type": "text", "text": "Create a short product description based on the provided #

```

Dataset Preparation:

The data needs to be converted into a format the model can understand. TRL supports popular dataset formats for instruction-based fine-tuning. For

[Open in app](#) ↗

Medium



Search



Write



The code returns a structured dictionary. It simulates a conversational setup where a “user” sends a message, and the “assistant” responds.

- **“role”: “user”:** This is the user’s input, where we provide a product name, category, and an image.

- **“role”: “assistant”**: This is the model’s response, which is the product description.
- The data is structured into a sequence of messages, which helps in creating vision-language prompts for the model.

4. Fine-Tuning VLMs Using TRL and SFTTrainer

Initialize the LLama-3.2 Vision Model

We will use the `AutoModelForVision2Seq` from Hugging Face’s Transformers library to load the model. We’ll also apply 4-bit quantization using `BitsAndBytesConfig` to save memory and boost performance. You can swap out this model for others like Qwen’s [Qwen2-VL-7B](#) or Mistral’s AI’s [Pixtral-12B](#).

To reduce memory usage while fine-tuning, we’ll use QLoRA (Quantized Low-Rank Adaptation), which quantizes the model to 4-bit precision, allowing it to run on smaller GPUs.

```
import torch
from transformers import AutoModelForVision2Seq, AutoProcessor, BitsAndBytesConfig

# Hugging Face model id
model_id = "meta-llama/Llama-3.2-11B-Vision-Instruct"

# BitsAndBytesConfig int-4 config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_use_double_quant=True, bnb_4bit_quant_type="nf4"
)

# Load model and tokenizer
model = AutoModelForVision2Seq.from_pretrained(
    model_id,
    device_map="auto",
    attn_implementation="flash_attention_2", # not supported for training
```

```

torch_dtype=torch.bfloat16,
quantization_config=bnb_config
)
processor = AutoProcessor.from_pretrained(model_id)

text = processor.apply_chat_template(
    dataset[2]["messages"], tokenize=False, add_generation_prompt=False
)

```



```

text = processor.apply_chat_template(
    dataset[2]["messages"], tokenize=False, add_generation_prompt=False
)
text

```

'<[begin_of_text]<[start_header_id]>user<[end_header_id]>\n\nCreate a Short Product description based on the provided ##PRODUCT NAME## and ##CATEGORY## and image.\nOnly return description, for a better mobile search experience.\n\n##PRODUCT NAME##: Barbie Fashionistas Doll Wear Your Heart\n##CATEGORY##: Toys & Games | Dolls & Accessories | Dolls<[image]><[eot_id]><[start_header_id]>assistant<[end_header_id]>\n\nyour style with Barbie Fashionistas Doll Wear Your Heart! This fashionable doll boasts a unique outfit and accessories, perfect for imaginative play. A great gift for kids aged 3+. Collect s #GirlsToys #FashionDoll #Play<[eot_id]>'

Explanation:

- **AutoModelForVision2Seq:** This model class from Hugging Face is specifically designed for Vision-to-Text tasks, where input images are mapped to text outputs.
- **AutoProcessor:** Prepares the input data for the model, handling tasks like tokenization and image pre-processing.
- **BitsAndBytesConfig:** Configuration for optimizing the model using 4-bit quantization, saving memory while keeping accuracy.
- **bnb_config:** Configures the model to run in 4-bit precision, which reduces the memory and computation requirements.
bnb_4bit_use_double_quant=True: Uses double quantization to maintain accuracy. **bnb_4bit_quant_type="nf4":** Specifies the type of quantization.
torch.bfloat16: Uses the bfloat16 data type for efficient computation.
- **device_map="auto":** Automatically maps the model across available GPUs.

Set Up LoRA Configuration for Fine-Tuning

We will fine-tune the model using LoRA (Low-Rank Adaptation) to optimize training speed and reduce memory consumption.

```
from peft import LoraConfig

# LoRA configuration based on QLoRA paper
peft_config = LoraConfig(
    lora_alpha=16,
    lora_dropout=0.05,
    r=8,
    bias="none",
    target_modules=["q_proj", "v_proj"],
    task_type="CAUSAL_LM"
)
```

Explanation:

- **lora_alpha=16:** Controls the rank of the low-rank matrices used in LoRA. Higher values increase capacity but may use more resources.
- **lora_dropout=0.05:** Adds some dropout during training to prevent overfitting.
- **r=8:** Rank of the low-rank approximation. Lower ranks reduce memory usage.
- **bias="none":** Specifies whether or not to apply bias correction.
- **target_modules=["q_proj", "v_proj"]:** Focuses the adaptation on specific layers of the model (query and value projections).
- **task_type="CAUSAL_LM":** Sets the task type to causal language modeling, where each word prediction is based on previously seen data.

Training Configuration

We'll use the `SFTTrainer` from the `trl` library to train the model with gradient checkpointing, a popular technique for saving GPU memory.

```
from trl import SFTConfig

args = SFTConfig(
    output_dir="fine-tuned-visionllama", # directory to save and repository id
    num_train_epochs=3,                  # number of training epochs
    per_device_train_batch_size=4,        # batch size per device during train
    gradient_accumulation_steps=8,        # number of steps before performing
    gradient_checkpointing=True,          # use gradient checkpointing to save
    optim="adamw_torch_fused",           # use fused adamw optimizer
    logging_steps=5,                     # log every 10 steps
    save_strategy="epoch",                # save checkpoint every epoch
    learning_rate=2e-4,                  # learning rate, based on QLoRA paper
    bf16=True,                           # use bfloat16 precision
    max_grad_norm=0.3,                   # max gradient norm based on QLoRA paper
    warmup_ratio=0.03,                   # warmup ratio based on QLoRA paper
    lr_scheduler_type="constant",         # use constant learning rate scheduler
    push_to_hub=True,                    # push model to hub
    report_to="tensorboard",              # report metrics to tensorboard
    gradient_checkpointing_kwargs = {"use_reentrant": False}, # use reentrant checkpointing
    dataset_text_field="",                # need a dummy field for collator
    dataset_kwargs = {"skip_prepare_dataset": True} # important for collator
)
args.remove_unused_columns=False
```

Explanation:

- **from trl import SFTConfig:** Imports the SFT (Supervised Fine-Tuning) configuration from the `trl` library.
- **output_dir="fine-tuned-visionllama":** The folder where the fine-tuned model will be saved.

- **num_train_epochs=3**: Specifies that the model will be trained for 3 epochs. Increase incase you are using for production or better accuracy.
- **per_device_train_batch_size=4**: Each GPU will handle a batch size of 4 samples during training.
- **gradient_accumulation_steps=8**: Gradients will accumulate over 8 steps before backpropagation, effectively simulating a larger batch size.
- **gradient_checkpointing=True**: Saves memory by not keeping intermediate values of activations.
- **optim="adamw_torch_fused"**: Uses the fused AdamW optimizer for efficient training.
- **logging_steps=5**: Logs the training metrics every 5 steps.
- **save_strategy="epoch"**: Saves the model at the end of every epoch.
- **learning_rate=2e-4**: Sets the learning rate for the optimizer.
- **bf16=True**: Uses the bfloat16 datatype to reduce memory usage.
- **max_grad_norm=0.3**: Caps the gradient norm to prevent exploding gradients.
- **warmup_ratio=0.03**: 3% of the training steps will be used for learning rate warmup.
- **lr_scheduler_type="constant"**: Uses a constant learning rate scheduler.
- **push_to_hub=True**: Automatically pushes the model to Hugging Face's model hub after training.
- **report_to="tensorboard"**: Logs the metrics to TensorBoard.
- **dataset_kwargs={"skip_prepare_dataset": True}**: Skips dataset preparation during training, it's already prepared.

Collate Data and Train the Model

Now, we need to prepare the data using the `collate_fn` function and initiate the training process. The `collate_fn` function is responsible for preparing a batch of data from the dataset in a format that the model can process during training. It takes a list of examples as input and returns a batch of tokenized text and processed image data, along with labels for supervised training.

```
from qwen_vl_utils import process_vision_info

# Define a custom collator function
def collate_fn(examples):
    texts = [processor.apply_chat_template(example["messages"], tokenize=False)
             for example in examples]
    image_inputs = [process_vision_info(example["messages"])[0] for example in examples]

    batch = processor(text=texts, images=image_inputs, return_tensors="pt", padding=True)
    labels = batch["input_ids"].clone()
    labels[labels == processor.tokenizer.pad_token_id] = -100

    # Mask image tokens in the labels
    image_tokens = [processor.tokenizer.convert_tokens_to_ids(processor.image_token)
                    for image_token_id in image_inputs]
    for image_token_id in image_tokens:
        labels[labels == image_token_id] = -100
    batch["labels"] = labels

    return batch

from trl import SFTTrainer

# Trainer setup
trainer = SFTTrainer(
    model=model,
    args=args,
    train_dataset=dataset,
    data_collator=collate_fn,
    tokenizer=processor.tokenizer,
    peft_config=peft_config
)

# Start training
trainer.train()
```

Start training our model by calling the `train()` method on our `Trainer` instance. This will start the training loop and train our model for 3 epochs. Since we are using a PEFT method, we will only save the adapted model weights and not the full model.

5. Testing the Fine-Tuned Model

Once the fine-tuning process is complete, you'll want to test how well your model performs. You can load the fine-tuned model and run it on new data, generating descriptions for new products:

```
from transformers import AutoModelForVision2Seq

# Load the fine-tuned model
model = AutoModelForVision2Seq.from_pretrained("output")
processor = AutoProcessor.from_pretrained(model_id)

# Prepare new product data
sample_product = {
    "product_name": "Hasbro Marvel Iron Man Action Figure",
    "category": "Toys & Games",
    "image": "https://example.com/product_image.jpg"
}

# Run inference
def generate_description(product, model, processor):
    # Prepare message
    messages = [
        {"role": "user", "content": [{"type": "image", "image": product["image"]}]}
    ]
    inputs = processor.apply_chat_template(messages, return_tensors="pt", padding=True)
    output = model.generate(**inputs)
    return processor.decode(output)

description = generate_description(sample_product, model, processor)
print(description)
```

[Google Colab Notebook Link](#)

[Watch this video for better understanding and how to run with GPU](#)

Conclusion

Fine-tuning multimodal models is great with specific task. After fine tuning you can use that model to build particular application. Whether you're building an e-commerce platform or tackling other multimodal challenges, this workflow simplifies the fine-tuning process and helps optimize your model's performance for real-world applications.

With tools like QLoRA and Hugging Face Hub, you can reduce memory usage and track your model's progress seamlessly. By following these steps, you can create more efficient, task-specific models that deliver better results in your chosen domain. I have build the [AI your fashion styling assistant](#). You can use llama 3.2 vision model in that project. I hope you learned and enjoyed this article. Follow me for more Generative AI content.

Artificial Intelligence

Fine Tuning

Llama 3

Generative Ai Tools

Vision



Written by Samar Singh

405 Followers · 11 Following

Follow

I Build Scalable AI Solutions 🚀 | Empowering Businesses with Intelligent Automation. Let's connect:- <https://www.linkedin.com/in/samar-singh-119855206>

Responses (2)



What are your thoughts?

Respond



Ilyes Ben Khalifa
about 2 months ago



Good job! I'm having an issue with the collate_fn function; it couldn't find 'messages' in the dataset.



1



1 reply

Reply



Anas Zafar
3 months ago



Nice article. I am facing an error while initializing the LLama3.2 model on your dataset

TemplateError: Prompting with images is incompatible with system messages.



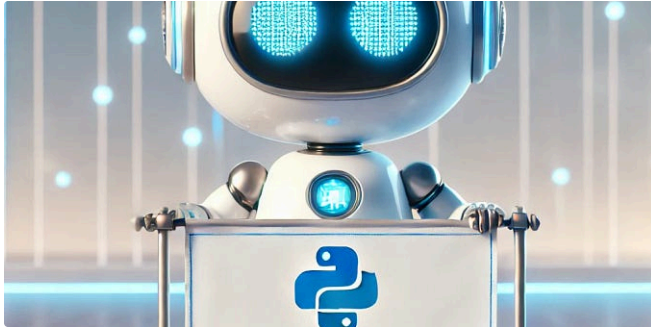
1



1 reply

Reply

More from Samar Singh

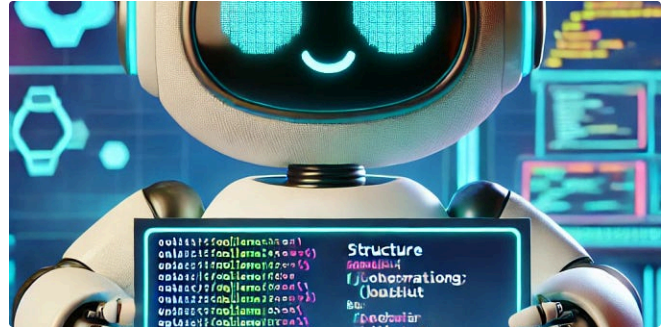


 Samar Singh

Pydantic AI: The Python Agent Framework to BUILD Production-...

Pydantic, a powerhouse in the Python ecosystem with over 285 million monthly...

★ Dec 8, 2024 🖱 185 💬 3 📌⁺ ⋮

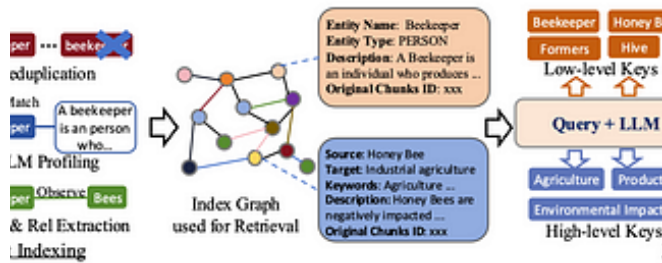


 Samar Singh

Ollama Introduces Structured Outputs: A Leap Towards Precisio...

Ollama, a pioneering platform in the AI space, has unveiled a transformative feature—...

★ Dec 10, 2024 🖱 126 📌⁺ ⋮



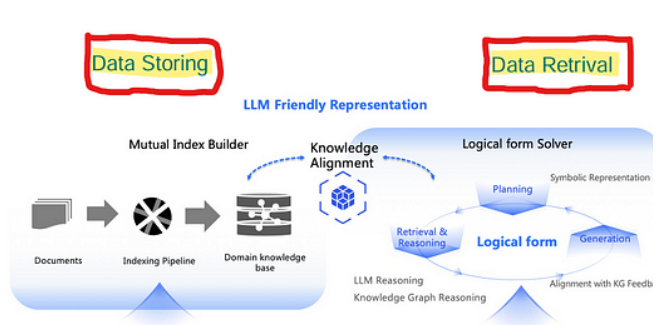
Overall architecture of the proposed LightRAG framework

 Samar Singh

LightRAG : A GraphRAG Alternative.

How to Set Up LightRAG Locally?

★ Oct 26, 2024 🖱 458 💬 1 📌 ⋮



 Samar Singh

Why Knowledge Augmented Generation (KAG) is the Best...

In the ever-evolving landscape of artificial intelligence, the introduction of Knowledge...

★ 5d ago 🖱 216 💬 1 📌 ⋮

See all from Samar Singh

Recommended from Medium





In Level Up Coding by Yeyu Huang



In Generative AI by Shobhit Agarwal

Build a Real-time AI Voice and Video Chat App with Function...

Gemini Development Tutorial V2



4d ago



343



3



Nov 10, 2024



522



Lists



AI Regulation

6 stories · 665 saves



Generative AI Recommended Reading

52 stories · 1580 saves



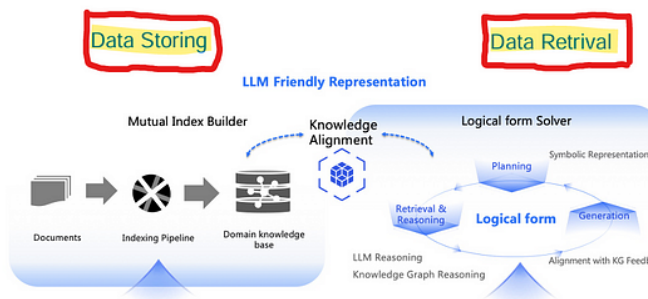
ChatGPT

21 stories · 930 saves



Natural Language Processing

1882 stories · 1516 saves



Samar Singh

Why Knowledge Augmented Generation (KAG) is the Best...

In the ever-evolving landscape of artificial intelligence, the introduction of Knowledge...



5d ago



216



1



Dec 11, 2024



11



VLM: {"res": "only json"}



MLX

RESPONSE ONLY IN JSON
Visual Language Models



alejandro

Structured Output in Local Vision Language Models (VLMs): A Step...

Implementing structured output in your small Visual Language Model (VLM) running on...

movie_title	revenue	review	genre
Shang-Chi	432.2	"solid film..."	Action
Titanic	2257.8	"still best..."	Romance
Titanic	2257.8	"a guilty..."	Romance
...	...	...	...



movie_title	revenue	review	genre
Titanic	2257.8	"still best..."	Romance
Titanic	2257.8	"a guilty..."	Romance
...	...	...	...

3

Answer Generation

"Summarize the reviews of the highest grossing romance movie considered a 'classic'."



In Artificial Intelligence in Plai... by Sreedevi Gog...



Venkat Ram Rao

From RAG to TAG: Leveraging the Power of Table-Augmented...

As artificial intelligence (AI) continues to evolve, so do the methods it uses to interact...



Dec 6, 2024



294



6



Jul 21, 2024



111



2



See more recommendations

Training a Mini(114M Parameter) Llama 3 like Model from Scratch

aka "How to Train you LLM"